

A Cloud-Native Security Architecture for Detecting Cyber Threats in Satellite Ground Segments

Space Ground Station Security Lab

<https://github.com/rikosoo/space-ground-station-security-lab>

Abstract

Satellite ground segments have become ordinary cloud workloads while retaining an extraordinary property: their actuator is a vehicle that cannot be rebooted, re-imaged, or taken offline. Security guidance for this domain is plentiful and largely qualitative; measurements of whether cloud-native security telemetry actually detects attacks against a ground segment are not. We ask a single question — *can cloud-native security telemetry effectively detect cyberattacks against satellite ground segments?* — and answer it with a reproducible testbed rather than an architecture diagram. The testbed simulates the full chain from spacecraft to incident response, executes five controlled attacks (credential compromise, unauthorized telecommand, telemetry tampering, telecommand replay, and mission-data exfiltration), and measures detection latency, false positives, defensive-control attribution, and cost. Across 100 trials all five attacks are detected, with a mean time to detect of 173.2s and 0.80 ± 0.05 false positives per day at 74.2% precision, for a modelled cost of approximately USD 419 per month for a single spacecraft. Three findings generalise beyond the testbed: detection latency is set by the tier a rule runs in rather than by the attack; cryptographic frame integrity and physics-aware analytics are complementary rather than redundant, because an adversary who disables the former is only visible to the latter; and reporting detection without reporting what the response failed to prevent overstates the security of the system. All code, detection content, labelled data and analysis scripts are released.

1 Introduction

A modern satellite ground segment is, from the outside, a fairly ordinary cloud workload: an authenticated web API, a message stream, an object store, and a handful of serverless functions. From the inside it is not ordinary at all. Its actuator is a spacecraft that cannot be rebooted, re-imaged, physically inspected, or taken offline for maintenance. It is reachable for a few minutes a day — in the configuration studied here, three passes totalling roughly 22 min out of every 24 h, leaving the adversary the other 97% of the day to work in. And the consequence of a successful attack is not data loss but the loss of a physical

asset in orbit.

The security guidance for this domain has grown quickly and is largely qualitative. Policy sets principles [10]; introductory guidance describes the problem space [8]; threat knowledge bases enumerate techniques [9]; the research literature has established that the attack surface is real, both by analysis [3, 4, 6] and by finding exploitable flaws in flight software actually used in orbit [12]. The 2022 KA-SAT incident [11] demonstrated that ground-segment compromise, rather than exotic attacks on the space segment, is the practical path to mission impact.

What is scarce is measurement. Given a ground segment instrumented the way a competent team would instrument any cloud workload — an audit trail, a streaming detection tier, findings aggregation, automated containment — how quickly are attacks actually detected? Which log makes each one visible? How much noise do the same detections produce during a week in which nothing happens? Which defensive mechanism did the work: the one that produced the alert, or one that quietly prevented the attack from mattering? And what does the whole arrangement cost to run?

Research question and sub-questions

RQ. Can cloud-native security telemetry effectively detect cyberattacks against satellite ground segments?

We decompose it into five measurable sub-questions:

RQ1 (Detection). Are the five attack classes detected, and how long does detection take relative to the first malicious action?

RQ2 (Noise). How many false positives do the same detections produce during benign operations, and at what precision?

RQ3 (Evidence). Which log source and which rule identify each attack?

RQ4 (Attribution). Which defensive mechanism prevents or limits each attack, as opposed to merely observing it?

RQ5 (Cost). What does the architecture cost per month, and how does that cost scale with fleet size?

Contributions

1. **A reproducible testbed** spanning spacecraft dynamics, RF link, ground station, mission-control API, cloud analytics and automated response, running deterministically on a virtual clock: fourteen simulated days in about two seconds, on a laptop, with no cloud account.
2. **Five controlled attack scenarios** with explicit adversary capability levels and per-event ground-truth labels, enabling true positives to be defined causally rather than by temporal coincidence.
3. **A detection pack of eleven rules** in two synchronised forms: executable Python that both the experiments and the deployed Lambda import, and portable Sigma [7] for other backends.
4. **An empirical evaluation** of detection latency, false positives, per-rule precision, a defence ablation that attributes outcomes to individual controls, a sensitivity analysis of the tuning parameters, and a cost model.
5. **A labelled dataset** and the full analysis pipeline, so every number in this paper can be regenerated with one command.

2 Background and related work

2.1 The space link and its security services

Telecommand and telemetry in most civil and commercial missions follow the CCSDS protocol stack: application data in Space Packets [1], carried in transfer frames, optionally protected by the Space Data Link Security protocol (SDLS) [2]. SDLS provides authentication, optional confidentiality, and — critically for this work — an anti-replay sequence number bound into the authenticated data. Our testbed models SDLS at the level of abstraction the detections care about: a keyed authentication tag over the frame payload and sequence number, and a sliding acceptance window on the vehicle. It does not model bit layouts, which do not change which events reach the analytics tier.

The two properties that matter for detection engineering are worth stating plainly. First, a frame that fails its tag check on the vehicle is an unambiguous signal: nominal operations never produce one, because the ground system signs every frame with the key the vehicle holds. Second, authentication alone does not establish freshness — a recorded frame remains cryptographically valid forever — which is why the anti-replay window, not the tag, is the control that defeats replay.

2.2 Threat knowledge for space systems

SPARTA [9] adapts the ATT&CK [5] structure to space systems, covering reconnaissance through impact for both the space and ground segments. We map every detection rule to a SPARTA technique and, where an analogue exists, to an ATT&CK technique; this makes the coverage claim checkable rather than rhetorical. NIST IR 8270 [8] and SPD-5 [10] supply the control expectations — authentication of commands, protection of telemetry integrity, monitoring — that the architecture implements.

2.3 Prior empirical work

Pavur and Martinovic [6] argue that satellite security research is held back by the absence of accessible testbeds, and that the field substitutes speculation for measurement as a result. Willbold et al. [12] answer part of that challenge on the space segment, analysing real satellite firmware and finding that several vehicles accept unauthenticated commands outright. Manulis et al. [4] survey the New Space threat landscape and identify the ground segment as the dominant practical attack surface, a judgement the KA-SAT incident [11] supports.

Our work is complementary and deliberately narrower. We do not analyse flight software. We assume the space segment implements its security services correctly and ask what the *ground* segment’s cloud telemetry sees when an adversary attacks the chain around it. To our knowledge this combination — an end-to-end ground-segment testbed, five labelled attack scenarios, and reported detection latency, false-positive rate, control attribution and cost — has not previously been published in a form a reader can rerun.

2.4 Why detection latency is the right metric here

In enterprise security, dwell time is measured in days and the marginal value of shaving seconds is low. A ground segment inverts this. The adversary’s window of *effect* is the pass: a command that cannot be transmitted cannot harm the vehicle. A pass in our configuration lasts about 7 min. A detection that fires in seconds can contain an intrusion before the next pass; one that fires in an hour cannot. Latency is therefore not a quality-of-service metric in this domain — it decides whether the response is capable of mattering.

3 Threat model

3.1 System under consideration

A single spacecraft in a 550 km, 53°-inclination orbit (period 95.7 min, eclipse fraction 0.37), operated through one ground station at -23.55° latitude with a 10° mask angle. This geometry yields three passes per day averaging

437s each. Mission control, the telemetry archive and all security analytics run in one cloud account.

3.2 Assets

- **Telecommand authority.** Direct control of a physical vehicle; worst case is loss of the spacecraft.
- **Frame authentication keys.** Possession allows indefinite forgery that no tag check can detect.
- **Telemetry integrity.** The danger is not the falsified number but the operator decision taken because of it.
- **The telemetry archive.** Payload products, orbit determination and mission intelligence.
- **The audit trail.** Cover for everything above.

3.3 Adversary capability levels

Each scenario declares which level it assumes, so that results are not implicitly claimed against a stronger or weaker adversary than the one executed.

N1 — remote, credentialed. Phishing, credential stuffing, relay of a second factor. No radio capability. Covers A1, A5 and the API portion of A2.

N2 — radio-capable. Can receive and transmit on the mission's bands and knows the frame format. Cannot break HMAC-SHA256. Covers A4 and the RF portion of A2.

N3 — ground-segment foothold. Controls a host between the demodulator and cloud ingestion. Covers A3.

Out of scope by assumption: an adversary who obtains the SDLS key, breaks the cryptography, or compromises the cloud provider's control plane. Each defeats the architecture by definition and would make the measurement uninformative rather than pessimistic. Also out of scope: jamming and other availability attacks, which are detected by link metrics rather than security telemetry, and whose inclusion would confound the false-positive measurement.

3.4 Assumptions

1. Frame authentication keys are generated in, and never leave, a managed key store.
2. The vehicle enforces its anti-replay window; the ground segment cannot persuade it to accept a stale frame.

3. Telemetry is archived before it is analysed, so detection never depends on the analytics tier being available in real time.
4. The detection pipeline is itself monitored. A silent pipeline is treated as an outage, not as a quiet week — an alarm on consumer lag is part of the architecture, not an operational nicety.
5. The operator population is small and enumerable. This is what makes network-origin profiling substantially more effective in a ground segment than in a general enterprise.

4 Architecture

4.1 Layers

The testbed implements five layers, each emitting events into one normalized schema:

1. **Spacecraft.** Circular-orbit propagation, power/thermal/attitude dynamics with physically bounded rates, a thirteen-command catalogue with a *critical* flag, SDLS tag verification and an anti-replay window.
2. **Ground station.** Pass geometry, an elevation-dependent frame error model, integrity verification of downlinked frames, and normalization of every observation into the event schema.
3. **Mission-control API.** Authentication with optional second factor, role-based command allowlists, per-principal rate limiting, and a pass-window feasibility check. Frames are signed here before transmission.
4. **Cloud.** A stream carrying normalized events, an encrypted object store for telemetry products, an audit trail covering both management and object data events, a streaming detection function, findings aggregation, metrics and alarms.
5. **Response.** A containment function subscribed to high and critical findings, implementing the containment step — and only the containment step — of the incident-response playbooks.

4.2 Two paths, two latencies

The architecture has exactly two detection paths and their latencies differ by an order of magnitude:

Stream path. Ground-segment observations (authentication, telecommands, frame integrity, telemetry samples) reach the detection function through the stream within seconds. Its bound is the consumer's batching window.

Control-plane path. Cloud API calls reach the same function through the audit trail and the event bus. Its bound is trail delivery, which is minutes, not seconds.

We model these as a per-rule *tier* (**stream** or **batch**) with a configurable latency added to every alert timestamp. Reporting a detection latency that excludes pipeline delay would flatter the architecture; making the delay an explicit, swept parameter (§7) makes the claim honest and lets a reader substitute latencies measured in their own account.

4.3 Normalized event schema

Every layer emits the same record: timestamp, source, event type, actor, target, outcome, source address, and a rule-specific attribute map. The schema is a small subset of the common event models used in practice — enough to write portable detections, small enough that its data dictionary fits on a page.

Each event additionally carries a ground-truth label (**benign** or an attack identifier). The label is written by the scenario, never read by a rule — a property enforced by a test that fails if the string appears anywhere in the rule module — and used only for scoring.

4.4 Deployment

The cloud instantiation is Terraform for a stream, an encrypted archive with lifecycle transitions, an audit trail with object-level data events on the archive, two functions, an event bus, findings aggregation, and least-privilege roles in which the detector can observe but not act and the responder can act only through a narrow set of reversible containment operations. The detection function imports the same rule package the experiments run, shipped as a layer: a single implementation is what allows results measured offline to be claimed for the deployed system.

4.5 Automated response, deliberately constrained

Only containment is automated, and only with reversible actions. A ground segment controls a vehicle that cannot be taken offline for maintenance; an automated action that safes a spacecraft in response to a false positive causes a real mission outage, and the response becomes the incident. The most aggressive action the automation can take is to stop the *ground segment* from transmitting. Nothing in the response path commands the spacecraft.

5 Detection content

Eleven rules cover the five scenarios. They fall into three kinds, and the distinction is not cosmetic — the kinds

Table 1: Detection content. Tier determines latency: **stream** rules see an event within seconds, **batch** rules cannot beat their own aggregation window.

ID	Kind	Tier	Detects
R01	thresholded	stream	A1
R02	behavioural	stream	A1, A2, A3
R03	policy	stream	A2
R04	behavioural	stream	A2
R05	policy	stream	A2, A4
R06	thresholded	stream	A3
R07	behavioural	batch	A3
R08	policy	stream	A4
R09	thresholded	batch	A5
R10	statistical	batch	A5
R11	policy	stream	A3, A5

behave differently under both attack and noise.

Policy rules fire on a violation of a control that nominal operations never produce: a command refused by an allowlist, a frame refused by the vehicle, an audit trail stopped. They are the cheapest and most precise rules in the pack and, in our results, the ones that carry most of the detection load.

Thresholded rules aggregate an individually noisy signal over a window: a burst of failed authentications, a cluster of frame integrity failures, a count of distinct objects read. Their threshold trades latency against noise directly.

Behavioural rules compare against a profile learned during a warm-up window: an unprofiled network origin, a principal issuing its first critical command, an hourly read volume far above baseline. They are the only rules that can catch an adversary who violates no policy.

Every rule maps to a SPARTA technique [9] and, where an analogue exists, to ATT&CK [5]. No attack depends on a single rule; that is a design requirement, and the ablation in §7 exists partly to check that removing one control does not open a blind spot.

5.1 Two rules worth describing in detail

R06, frame integrity, and why it is thresholded.

A radio link corrupts frames on its own, particularly at low elevation at the edges of a pass. In our model the frame error probability rises exponentially as elevation falls, producing roughly six to nine corrupted frames per week out of about 1800. A rule that alerts on a *single* failed tag detects tampering approximately 60s sooner and generates enough noise to be switched off within a week — the exact failure mode a detection architecture exists to avoid. R06 therefore requires three failures within 120s. §7 quantifies both sides of that trade.

R07, physics, and the rail exclusion. R07 checks three things about each telemetry channel: that it lies inside its physical envelope, that it is not changing faster than the subsystem can change, and that it is not frozen. The third check is what catches a masking attack that holds a degrading measurement constant. Its first implementation was the largest single source of false positives in the testbed, because a fully charged battery in sunlight is legitimately constant. Excluding channels resting against a physical rail removed that entire class. The lesson generalises beyond this testbed: a stuck-value detector on any physical process must know which constants are physics and which are faults.

Analytics should not run on bits already known to be corrupt. R07’s second-largest false-positive source was subtler than the rail problem and more instructive. A corrupted frame contains implausible values by definition, so the physics rule dutifully alerted on link noise that the integrity check had *already* rejected — two rules alarming on one event, one of them uselessly. Excluding samples whose frame failed its tag check removed every such alert and left a clean division of labour: R06 owns provenance failures, R07 owns content. The general form of the lesson is that a detection pipeline should establish trust in an input before reasoning about its meaning.

5.2 Portability

Each rule exists in two synchronised forms: executable Python, which both the experiments and the deployed function import, and Sigma [7], which can be converted for another backend. A test asserts that the two sets of rule identifiers are equal, so the packs cannot silently drift apart — a failure mode common enough in practice to be worth automating against.

6 Methodology

6.1 Protocol

One trial has two phases:

1. **Warm-up** (7 simulated days). Nominal operations with the engine in learning mode. Behavioural rules build their profiles and emit nothing.
2. **Measurement** (7 simulated days, 5,823 events). Nominal operations continue while the five attacks are injected at staggered start times, separated by 20h plus a per-trial random jitter of up to 8h. The spacing makes every alert attributable to at most one attack; the jitter prevents each trial from placing attacks at the same point of the orbital cycle, which would report schedule artefacts as variance.

We run 100 independent trials with different seeds and report means with 95 % normal-approximation confidence intervals.

6.2 The benign workload

A testbed with no benign anomalies reports a false-positive rate of zero and says nothing useful. Ours generates four classes of benign noise, chosen because each is a real analyst’s daily experience: operators mistyping passwords (occasionally enough times in a row to resemble a spray); operators connecting from mobile or travel networks; the radio link corrupting frames at low elevation; and a legitimate weekly reprocessing job that bulk-reads the archive in a pattern superficially identical to exfiltration. Their *rates* are chosen, not measured from a real mission — a limitation we return to in §9.

6.3 Scoring definitions

True positive. An alert at least one of whose evidence events carries a non-benign ground-truth label. Attribution therefore follows causality, not temporal coincidence: an alert that merely fires during an attack window, caused by unrelated benign activity, counts as a false positive.

Time to detect (TTD). The first attributed alert’s actionable timestamp minus the attack’s first malicious action. It includes the pipeline latency of the tier that produced the alert. For the replay scenario the reference point is the first re-transmission, not the passive radio capture, because capture leaves no observable in the ground segment.

Time to contain (TTC). The first automated containment action on the attack’s entity minus the same reference point.

Impact. Malicious actions that took effect — commands executed, objects read, forged frames accepted — reported both in total and, separately, as critical commands executed and megabytes exfiltrated, because “one executed thruster firing” and “one object read” are not comparable harms.

Impact after containment. The subset of the above occurring after the first containment action: what the response did *not* prevent.

6.4 Deduplication as part of the system under test

Without a per-entity cooldown, a single incident produces an alert storm and the false-positive rate becomes meaningless. The cooldown (300s by default) is therefore an explicit, swept parameter rather than a hidden implementation detail.

6.5 Ablation and sensitivity

The ablation repeats the whole campaign with one control disabled at a time across seven preventive and responsive mechanisms, plus an all-disabled configuration. The sensitivity analysis sweeps four tuning parameters — the frame-integrity threshold, the batch-tier latency, the archive enumeration threshold and the deduplication cooldown — one at a time with everything else at default.

6.6 Reproducibility

Every run is seeded and deterministic; a test asserts that two trials with the same seed produce identical latencies and false-positive counts. Fourteen simulated days take about two seconds of wall time. Every number in this paper is generated from the experiment JSON by a script that emits the LaTeX macros and table bodies the document includes, so a stale figure in the text is not possible.

7 Results

7.1 RQ1 — detection and latency

All five attacks were detected in all 100 trials (Table 2). Mean time to detect across the campaign is 173.2 s.

The striking feature is not the latency but its *variance*, and what governs it. For four of the five attacks the distribution is tight across trials with randomised start times: A4 is identical in every trial, and A1, A3 and A5 vary by ± 0.4 s, ± 0.3 s and ± 1.9 s about means of tens to hundreds of seconds. For those four, latency is a property of the architecture rather than of the adversary, and it decomposes cleanly into two terms.

A2 is the exception, and it is the more instructive case. Its mean of 558.5 s describes no trial that occurred, because the distribution is bimodal. In 98 of 100 trials the hijacked session is observable, R02 fires on it, and the attack is detected in 4.0 s. In the remaining 2 the session never materialises: the operator account A2 targets was still blocked by an earlier automated response — usually one a false positive triggered within the preceding 24 hours — and the API refuses the hijack without emitting an authentication event at all. With no logon to reason about, R02 has nothing to evaluate, and detection falls through to R05 on the RF phase, which can only fire while the spacecraft is in a pass window. Latency is then set by orbital mechanics rather than by the pipeline, reaching 49534.0 s.

Two things follow. First, a campaign of ten trials has a substantial chance of drawing none of the slow mode and reporting a confidence interval of zero for A2, which is why the baseline here runs 100. Second, the second mode is not an adversary capability: it is an interaction between the response system and the detection system, in which containing one attack degrades the observability of the next. We return to this in Section 8.

The first term is the pipeline. A2 (in its fast mode), A3 and A4 are detected in 4.0 s, 13.6 s and 4.0 s respectively — essentially the stream-path latency itself — because their first malicious action is already sufficient evidence: a session from an unprofiled origin, a frame that failed its tag, a duplicated frame counter. Nothing needs to accumulate.

The second term is the rule’s own threshold, and it dominates whenever evidence must accumulate. A1 is detected at 47.8 s not because the pipeline is slow but because R01 waits for five failed authentications, and the adversary sprays one every eleven seconds. A5 is detected at 242.2 s because R09 waits for twenty-five distinct objects at one every four seconds. In both cases the SOC could halve the latency by halving the threshold, and would pay for it in noise — which is precisely the trade the sensitivity analysis below quantifies.

7.2 RQ2 — false positives

Across the measurement windows the pack produced 0.80 ± 0.05 false positives per day at 74.2 % precision (Table 3). For a single spacecraft that is 5.6 alerts a week an analyst will close as benign. Whether this is acceptable is not a question about precision in the abstract: it is a question about whether a team that already runs passes can absorb about one extra triage per weekday, and at this rate it plainly can.

That figure depends on something easy to overlook, which we report because it changed our own numbers. Automated containment blocks the principal an alert names, including when the alert is wrong. If the block never lifts, the benign account it hit stops producing events altogether, and an account that produces no events cannot produce another false positive. The response was therefore suppressing the very quantity we were trying to measure. With blocks that expire after 24 hours — the configuration reported here, and the realistic one, since a wrongly suspended operator is restored after triage — the rate is 0.86 per day; with blocks that never lift it is 0.50, for the same prevention outcome (51.7 versus 52.0 adversary actions taking effect). Roughly two in five of the false positives the pack really produces were hidden by permanent suspension, not avoided by the rules.

The distribution matters more than the total. Policy rules — a command outside an allowlist, a frame the vehicle refused, an audit trail stopped — produced no false positives whatsoever, because in a correctly configured ground segment nominal operations genuinely never generate those events. All false positives came from the thresholded and behavioural rules, and they came from precisely the benign anomalies the workload was built to contain: an operator whose typos crossed the brute-force threshold, and the weekly reprocessing job, whose access pattern R09 cannot distinguish from exfiltration on volume alone.

R10, the archive volume anomaly, is worth a paragraph

Table 2: Per-attack detection performance over 100 trials. TTD and TTC are measured from the first malicious action and include pipeline latency. “Actions that succeeded” counts adversary actions that took effect, whether or not they were detected. Median and p95 are given alongside the mean because A2’s latency is bimodal, and for that attack the mean describes no trial that actually occurred; its maximum is 49534.0 s.

Attack	Det. (%)	Mean TTD (s)	95 % CI	Median TTD (s)	p95 TTD (s)	Mean TTC (s)	First rule	Actions succeeded
A1	100	47.8	± 0.4	48.0	48.0	67.8	R01	0.0
A2	100	558.5	± 976.5	4.0	4.0	598.1	R02, R05	0.0
A3	100	13.6	± 0.3	14.0	14.0	33.5	R06	27.9
A4	100	4.0	± 0.0	4.0	4.0	24.0	R08	0.0
A5	100	242.2	± 1.9	246.0	246.0	263.0	R09, R10	24.2

Table 3: Per-rule alert quality, summed over all trials. Policy rules produce no false positives at all; every false positive in the pack comes from a thresholded or behavioural rule.

ID	Rule	TP	FP	Prec.	FP/day
R01	Brute force then success	100	31	76	0.04
R02	Unprofiled origin	193	27	88	0.04
R03	RBAC violation	98	0	100	0.00
R04	Unusual critical cmd	294	17	95	0.02
R05	Uplink auth failure	100	0	100	0.00
R06	Frame integrity cluster	105	4	96	0.01
R07	Physics violation	92	380	19	0.54
R08	Replayed frame	200	0	100	0.00
R09	Archive enumeration	100	100	50	0.14
R10	Archive volume anomaly	100	0	100	0.00
R11	Control-plane change	195	0	100	0.00

for a different reason. It corroborates A5 in every trial at no cost in noise (Table 3), but it does so only after a defect we found by asking why it had previously fired almost never. The rule aggregates bytes read into hourly buckets, and the implementation closed a bucket when the *next* read from that principal arrived past the hour boundary. That is sound while traffic continues and silently wrong the moment it stops — and stopping it is precisely what containment does. Once R09 fired and the responder denied the principal, no further read ever arrived, so the bucket holding the exfiltration was never evaluated at all. The rule was not insensitive to the burst; it was never asked about it.

Closing buckets on the clock instead, which is what the Sigma form of the rule (`timeframe: 1h`) always specified and what a scheduled analytics job does in the deployed system, makes the rule report the burst in every trial without adding a single false positive. We record the defect rather than only the corrected number because it is an instance of the interaction that Section 8 draws out: a detection whose evaluation is driven by the arrival of the traffic it inspects can be silenced by any control that stops that traffic, and the silence is indistinguishable from an absence of evidence. It also changes what R10 is for. It is not a redundant second opinion on A5 — R09 is still the faster detector, and detects first in almost every trial — but an independent one that survives the case an object-count threshold cannot see: an adversary who reads a small number of very large objects.

Table 4: First evidence by log source. Sources marked *ground* are produced by the ground segment itself and must be shipped deliberately.

Attack	First evidence	Origin
A1	<code>api.auth</code> failures + success	ground
A2	<code>api.auth</code> session origin, then <code>gs.uplink</code>	ground
A3	<code>gs.downlink</code> integrity failures	ground
A4	<code>gs.uplink</code> replay rejection	ground
A5	<code>cloud.data</code> object reads	cloud

R07’s remaining false positives are the most interesting result in this table, because the attack causes them. Once A3 disables frame authentication, every subsequent frame arrives unauthenticated, and the ground segment loses its ability to tell link corruption from injection. The physics rule — now the only detection left — therefore inherits the radio’s noise and starts alerting on naturally corrupted frames. Disabling frame authentication does not merely remove a detection; it degrades the precision of the detection that replaces it. We report this rather than suppressing it, because a reader deploying content analytics as a fallback for cryptographic integrity should know that the fallback is noisier precisely when it is needed.

The reprocessing-job case deserves a direct statement rather than a footnote. The correct remedy is to exclude the reprocessing principal by identity and alert instead when *that* principal reads from an unexpected origin. Raising the threshold until the job stops alerting — the tempting fix — would also raise it past the attack, which is how detections quietly stop working.

7.3 RQ3 — which logs identify each attack

Table 4 maps each attack to the log source that carried its first evidence. No attack was visible in only one source, but the sources are not interchangeable: three of the five were first identified by ground-segment logs that no cloud-native service produces by default. A deployment that instruments only the cloud control plane would have detected A1 and A5, would have detected A3 only at its second phase, and would have missed A2’s radio variant and A4 entirely.

7.4 RQ4 — which defence did the work

Table 5 repeats the campaign with one control removed at a time. Detection rate is not the interesting column: every configuration still detects every attack, which is the defence-in-depth property the pack was designed for. Impact is the interesting column.

Three results stand out.

Frame authentication and the anti-replay window are the only controls that protect the vehicle. With the full stack, 0.9 critical commands reach the spacecraft on average — these are A3’s TLM_AUTH_DISABLE, issued through a hijacked flight-director session and therefore inside the allowlist, which is exactly the case an entitlement check cannot catch. Removing frame authentication raises the figure to 2.9; removing the anti-replay window raises it to 0.9. Nothing in the cloud analytics tier substitutes for either. Detection still fires — promptly — but the command has already executed on a physical vehicle when it does. This is the clearest instance in our data of the general principle that on a system with a physical actuator, detection is not a substitute for prevention.

Automated containment prevents most of the impact. Disabling it raises total successful malicious actions from 52.0 to 288.0 and exfiltrated volume from 1,218 MB to 13,086 MB, with identical detection latency. The alerts were the same; the difference is entirely in whether anything acted on them within 20s.

Letting blocks expire costs precision and buys back observability. The block-expiry arm is the only one in the table where a control’s removal *improves* a reported metric: with blocks that never lift, false positives fall from 0.86 to 0.50 per day while prevention is unchanged (52.0 versus 51.7 actions taking effect). That is not a case for permanent blocks. It is the measurement artefact described in Section 7, seen from the other side: a suspended benign account emits nothing further, so it neither trips a rule again nor appears in the denominator of anything. What the expiry buys in exchange is the observability discussed above — A2’s slow mode, in which an already-blocked account makes the adversary’s session invisible, becomes rare rather than routine. The choice between the two is a real operational trade-off, and a pack that reports only the permanent-block number is reporting the flattering half of it.

A control can be redundant because another one already covered it. Disabling MFA changed no outcome at all, which is initially surprising for the one control that genuinely prevents A1. The explanation is in the timeline: R01 fires on the fifth failed authentication and containment revokes the account before the adversary’s spray reaches the correct password. Detection substituted

for prevention here — but only because the adversary was noisy. In the all-disabled configuration, where nothing contains the account either, A1 becomes the second most productive attack in the campaign. Redundancy between a preventive and a detective control is worth having and worth knowing about; it is not a reason to remove either.

Some controls cost nothing and are worth having anyway. Removing the rate limit or the pass-window feasibility check changed no outcome in our scenarios. We report this rather than omitting it: their value in this testbed is that they force an adversary into behaviour that is easier to detect, not that they block anything on their own.

7.5 Sensitivity of the tuning parameters

Table 6 shows the frame-integrity threshold trade-off directly. At a threshold of one the rule detects tampering 60.6s on average but produces 1.41 false positives per day; at three it detects at 62.6s with 0.86 per day. Buying roughly a minute of latency in exchange for a rule an analyst will not mute is, in our judgement, the right trade — but it is a judgement, and the table is what makes it a judgement rather than a guess.

The batch-tier latency sweep is starker still: moving from a 30s to a 900s query cadence moves campaign mean TTD from 38.6s to 212.6s while changing the false-positive rate not at all. Query cadence is free latency — the most consequential and most easily overlooked parameter in the architecture.

7.6 RQ5 — cost

Table 8 gives the modelled monthly bill for the measured workload: approximately USD 419 for one spacecraft and one ground station, of which 82 % is OpenSearch Serverless.

The scaling behaviour (Table 9) is the practically important result: per-spacecraft cost falls to USD 79.89 at a hundred vehicles. At lab scale the bill is dominated by the minimum billable capacity of the search tier rather than by event volume — the architecture is not expensive because it processes much, but because it must exist at all. For a single-satellite academic or startup mission this is the dominant economic fact, and it argues for pooling the analytics tier across missions rather than for reducing telemetry.

8 Discussion

8.1 Answering the research question

Cloud-native security telemetry does detect all five attack classes, quickly and at a false-positive rate a small team can absorb — but the qualifier matters more than the headline. Three of the five attacks were first identified

Table 5: Defence ablation, 100 trials per configuration. Detection survives every single-control removal; impact does not.

Configuration	Det. (%)	TTD (s)	Actions	Critical cmds	MB out	FP/day
Full stack	100	62.6	52.0	0.9	1,218	0.86
No MFA	100	62.6	52.0	0.9	1,218	0.86
No RBAC	100	62.6	54.0	1.9	1,218	0.89
No frame authentication	100	62.6	54.0	2.9	1,218	0.86
No anti-replay window	100	62.6	55.0	0.9	1,218	0.84
No rate limit	100	62.6	52.0	0.9	1,218	0.86
No pass-window check	100	62.6	52.0	0.9	1,218	0.86
No automated containment	100	62.5	288.0	1.0	13,086	1.00
Blocks never expire	100	62.5	51.7	0.8	1,213	0.50
No controls	100	62.5	306.0	6.6	13,086	1.14

Table 6: Frame-integrity threshold: latency against noise.

Threshold	Mean TTD (s)	Det. (%)	FP/day	Prec. (%)
1	60.6	100	1.41	63
2	61.6	100	0.90	72
3	62.6	100	0.86	73
5	64.6	100	0.84	73
8	4281.3	100	0.84	72

Table 7: Batch-tier query cadence. Latency is bought with money, not with noise.

Latency (s)	Mean TTD (s)	Det. (%)	FP/day	Prec. (%)
30.0	38.6	100	0.86	73
150.0	62.6	100	0.86	73
300.0	92.6	100	0.86	73
900.0	212.6	100	0.86	73

by logs the *ground segment* produces, not by anything a cloud provider generates on its own. The cloud tier supplied the pipeline, the correlation, the findings store and the response automation; it did not supply the evidence. An organisation that adopts cloud-native security tooling and instruments only what that tooling emits by default will detect credential abuse and data exfiltration, and will be blind to the attacks that reach the vehicle.

The honest form of the answer is therefore conditional: cloud-native security telemetry is effective for ground segments *if the ground segment is instrumented as a first-class log source*. That conclusion is unglamorous and, in our view, the most useful thing this study produces.

8.2 Detection is not a substitute for prevention

The ablation makes this concrete in a way that a narrative argument cannot. Removing frame authentication does not reduce the detection rate at all — every forged command is still detected, promptly — yet it raises the number of critical commands executing on the spacecraft from 0.9 to 2.9. A perfect detector and a 4 s pipeline do not undo a thruster firing.

This is a general property of systems with physical ac-

Table 8: Modelled monthly cost, single spacecraft, on-demand list prices, no committed-use discounts. Lines below USD 0.01 omitted.

Service	Cost driver	USD/month
OpenSearch Serverless	indexing + search	345.60
Kinesis Data Streams	stream-hours	28.80
S3	archive storage	23.29
CloudWatch	alarms, metrics	14.50
Security Hub	findings ingested	5.00
KMS	key + requests	2.07
EventBridge	rule matches	0.02
GuardDuty	data analysed	0.02
Lambda	detection invocations	0.01
S3	PUT/GET requests	0.01
CloudWatch Logs	log ingestion	0.01
CloudTrail	S3 data events	0.01
Total		419.37

Table 9: Cost scaling. The architecture is a fixed platform cost at small scale.

Spacecraft	Events/day	USD/month	USD/spacecraft
10	8,318	1,660.81	166.08
100	83,182	7,989.42	79.89

tuators, and it argues for a specific division of labour. Preventive controls belong as close to the actuator as possible — on the vehicle, where the anti-replay window lives, not in the analytics tier. Detection exists to catch what prevention does not cover and to bound the duration of an intrusion, not to replace it. In a ground segment the distinction is unusually sharp, because the window in which prevention can act is the pass, and the pass is over in seven minutes.

8.3 Cryptographic integrity and content analytics are complementary

Attack A3 was designed to separate two defences that are often treated as alternatives. Its first phase injects implausible values with a forged tag: the cryptographic check catches it immediately. Its second phase first disables frame authentication and then freezes a degrading

measurement at a plausible constant. After that point the tag check has nothing to say — every frame is “correctly” unauthenticated — and the only detection left is the one that reasons about whether the measurement makes physical sense.

The measurement adds a wrinkle we did not anticipate. Once authentication is off, the physics rule also inherits the link’s own noise, because the ground segment can no longer separate a corrupted frame from an injected one. The fallback detection is therefore least precise exactly when it becomes the only one — a property worth knowing before treating content analytics as a substitute for provenance.

Neither defence subsumes the other. Cryptography establishes provenance and says nothing about content; physics analytics evaluates content and says nothing about provenance. A ground segment that has only one of them has a documented blind spot, and A3 traverses it in two moves.

8.4 Where the false positives came from

Every false positive in the campaign came from a thresholded or behavioural rule, and each traced to a benign anomaly that a real ground segment produces every week. This has a practical implication for anyone building such a pack: the policy rules — refused commands, refused frames, stopped audit trails — are close to free, and should be written first. They are precise because the events they match genuinely do not occur in nominal operations, and that property comes from the ground segment’s small, well-defined operational envelope rather than from clever rule authoring.

The behavioural rules earn their noise, though. R02, the unprofiled-origin rule, was the first detector for two of the five attacks. In a general enterprise, origin profiling is a notoriously noisy control; in a ground segment, where the operator population is a handful of named people working from a handful of known networks, it is one of the strongest signals available. Threat detection content does not transfer between domains unchanged, and this is a case where the domain makes a weak enterprise rule into a strong one.

8.5 Containment degrades the observability of later attacks

The A2 latency distribution in Section 7 is bimodal for a reason that has nothing to do with the adversary. Automated containment blocks a principal, and in this architecture a block is permanent for the remainder of the campaign. When a false positive blocks a benign operator — which happens at the false-positive rate the pack already reports — that account stops being usable, and the mission-control API refuses any later session for it *silently*, without emitting an authentication event. An adversary who subsequently targets that account there-

fore produces no logon for the behavioural rules to evaluate. Detection does not fail, but it falls back to a slower path: in our campaign, from 4.0s on the session to as much as 49534.0s on the radio-frequency phase, which can only be observed during a pass.

This is worth stating plainly because it inverts an assumption that is easy to make. Automated response is normally evaluated on what it prevents, and Section 7 reports that in those terms. But a response action also changes what the telemetry contains, and a control that removes an actor from the system removes that actor’s events along with it. The effect is not confined to this testbed: any architecture that disables accounts automatically and logs refusals at a different fidelity than successes will have the same blind spot. Two mitigations follow directly. The first is to expire automated blocks rather than leaving them indefinite, which is what this testbed now does: a block lifts after 24 hours, modelling a suspension an analyst releases after triage rather than an account that is deleted. It is not free: Table 5 contrasts it with an arm in which blocks never lift, and the expiry raises false positives from 0.50 to 0.86 per day, because a re-stored account can trip a rule again where a suspended one never could. We take that trade as the right one, and report both halves of it. The second is to emit an explicit event when an action is refused because the principal is blocked, so that the refusal is itself a signal rather than a silence. We have not implemented that one, and it is the more general fix: it would make the blind spot visible even where a block is genuinely permanent.

8.6 Implications for practitioners

1. **Ship ground-segment logs.** Authentication, telecommand submission, uplink verdicts and frame integrity are the evidence for three of five attacks. Nothing generates them for you.
2. **Put the anti-replay window on the vehicle and verify it.** It is the only control in the study that both prevents an attack and produces its alert.
3. **Choose your query cadence deliberately.** Moving batch-tier latency from 900s to 30s improved campaign mean latency by more than any rule change we made, at no cost in false positives.
4. **Exclude known-good service principals by identity, never by threshold.** The weekly reprocessing job and the exfiltration attack are indistinguishable on volume; raising the threshold past the job also raises it past the attack.
5. **Automate containment, not remediation.** Containment cut impact by roughly four fifths in our data. Automating anything that commands the vehicle would make a false positive into a mission outage.

6. **Budget for the platform, not the volume.** At single-mission scale the cost is a floor, not a function of telemetry. Pool the analytics tier across missions rather than reducing what you collect.

9 Limitations and threats to validity

9.1 Construct validity

The most significant limitation is that the offline harness *models* pipeline latency rather than measuring it in a live cloud account. Our stream-path figure of 4s and batch-path figure of 150s are plausible for the services in question but they are parameters, not observations. We mitigate this in three ways: the parameters are named rather than buried; the sensitivity analysis reports results across a wide range of them; and the deployed functions import the identical rule package, so a reader with an account can substitute measured latencies without changing the detection logic. Absolute latencies should nonetheless be read as *lower bounds conditioned on the modelled pipeline*, and the relative comparisons — stream against batch, threshold against threshold — are the durable results.

The impact measure is coarse. Counting “actions that succeeded” treats an executed thruster firing and a read object as one unit each, which is why we report critical commands and exfiltrated volume separately. A severity-weighted composite would be more informative and would also be more arbitrary; we chose transparency.

9.2 Internal validity

Ground-truth labels are written by the attack scenarios themselves, so an error in a scenario would produce a systematically wrong score. We guard against the most dangerous form of this — a rule reading the label it is being scored against — with a test that fails if the label appears anywhere in the rule module. Attribution by evidence causality rather than by time window removes the other common failure, in which unrelated benign alerts during an attack window are counted as detections.

Attacks are separated by at least 20h so that alerts attribute cleanly. Real intrusions overlap, and a campaign of simultaneous attacks would stress deduplication and analyst triage in ways this design does not measure.

9.3 External validity

The benign workload is synthetic. Its noise sources are realistic in kind — typos, roaming operators, link fades, a weekly bulk job — but their rates are chosen. The false-positive figures should therefore be read as *order-of-magnitude* results conditioned on that workload, not as a prediction for a specific mission. A team adopting

this pack should expect to re-tune thresholds against their own traffic; the sweep tables show how much movement each threshold buys.

The configuration is one spacecraft, one ground station, one cloud account. Multi-tenant ground-station-as-a-service, cross-mission analytics, and constellation-scale operations each introduce isolation and correlation questions this study does not address.

The spacecraft model is deliberately shallow: circular two-body propagation, no perturbations, no manoeuvre dynamics, no link budget in decibels. This is sufficient because the detections depend on pass timing, physical envelopes and frame outcomes, all of which the model reproduces — but it means the testbed cannot be used to study attacks whose detection depends on finer physics, such as subtle orbit-determination manipulation.

9.4 What we did not evaluate

Availability attacks (jamming, uplink flooding) are excluded because they are detected by link metrics rather than security telemetry. Supply-chain compromise of flight software, physical attacks on the ground station, and adversaries who obtain the SDLS key are excluded by the threat model. Analyst experience — the question of whether 0.80 false positives per day is actually tolerable in a given SOC — is a human-factors question our method cannot answer.

10 Conclusion

We built a reproducible testbed spanning the full satellite ground segment, executed five controlled attacks against it, and measured what a cloud-native security architecture actually sees. All five attacks were detected across 100 trials, with a campaign mean time to detect of 173.2s, 0.80 ± 0.05 false positives per day, 74.2% precision, and a modelled cost of approximately USD 419 per month for a single spacecraft, falling to USD 79.89 per spacecraft at a hundred.

The answer to the research question is a qualified yes. Cloud-native security telemetry detects these attacks well, but the cloud tier contributes the pipeline, the correlation and the response — not the evidence. Three of the five attacks are visible only in logs the ground segment must be instrumented to produce. Detection latency is set by which tier a rule runs in rather than by adversary behaviour, so architectural placement dominates rule tuning. Cryptographic frame integrity and physics-aware analytics defend against different halves of a tampering attack and neither substitutes for the other. And on a system whose actuator is a spacecraft, detection does not substitute for prevention: removing frame authentication changed our detection rate not at all while allowing critical commands to execute on the vehicle.

Future work

Four directions follow directly. First, replace the modelled pipeline latencies with measurements from a deployed account, which the shared rule package makes a matter of instrumentation rather than reimplementation. Second, replace the synthetic benign workload with traces from an operating mission — the single change that would most improve the external validity of the false-positive results. Third, extend to constellation scale and to multi-tenant ground-station-as-a-service, where cross-mission correlation becomes possible and tenant isolation becomes a new attack surface. Fourth, study overlapping and adaptive adversaries, including one that observes the containment response and adjusts — a case the current staggered design deliberately excludes.

Availability

The testbed, the eleven detection rules in both executable and Sigma form, the five attack scenarios, the incident-response playbooks, the Terraform for the cloud deployment, the labelled event dataset and the analysis scripts that generate every number in this paper are released at <https://github.com/rikosoo/space-ground-station-security-lab>. All results in this paper are regenerated by a single command and are deterministic under a fixed seed.

References

- [1] Consultative Committee for Space Data Systems. Space packet protocol. Recommended Standard (Blue Book) CCSDS 133.0-B-2, CCSDS, 2020.
- [2] Consultative Committee for Space Data Systems. Space data link security protocol. Recommended Standard (Blue Book) CCSDS 355.0-B-2, CCSDS, 2022.
- [3] Gregory Falco. Cybersecurity principles for space systems. *Journal of Aerospace Information Systems*, 16(2):61–70, 2019.
- [4] Mark Manulis, Christopher P. Bridges, Richard Harrison, Venkatesh Sekar, and Andy Davis. Cyber security in new space: Analysis of threats, key enabling technologies and challenges. *International Journal of Information Security*, 20:287–311, 2021.
- [5] MITRE. MITRE ATT&CK enterprise matrix, 2024. <https://attack.mitre.org/>.
- [6] James Pavur and Ivan Martinovic. SOK: Building a launchpad for impactful satellite cyber-security research, 2020.
- [7] Florian Roth, Thomas Patzke, et al. Sigma: Generic signature format for SIEM systems, 2024. <https://github.com/SigmaHQ/sigma>.
- [8] Matthew Scholl and Theresa Suloway. Introduction to cybersecurity for commercial satellite operations. Technical Report NIST Internal Report 8270, National Institute of Standards and Technology, 2023.
- [9] The Aerospace Corporation. SPARTA: Space attack research and tactic analysis, 2024. <https://sparta.aerospace.org/>.
- [10] The White House. Space policy directive-5: Cybersecurity principles for space systems, 2020.
- [11] Viasat. KA-SAT network cyber attack overview, 2022. Public incident statement, 30 March 2022.
- [12] Johannes Willbold, Moritz Schloegel, Manuel Vögele, Maximilian Gerhardt, Thorsten Holz, and Ali Abbasi. Space odyssey: An experimental software security analysis of satellites. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 1–19, 2023.